Named Pipes

Proyecto DI_PSP

La Salle Santo Ángel

2026/27

Alumnos:
Dumitru, Neli Alexandra
Vilellas Martín, Raúl

Profesor:
Villanueva Fernández, Javier

Índice

1. NAMED PIPELINES en C#.....3

Cómo funciona..... 3

Clases que utilizaremos en C#..... 3

2. DOCUMENTACIÓN..... 4

3. USO EN EL PROYECTO..... 5

Responsabilidad de cada aplicación..... 5

Pasos de una medición..... 5

4. DATOS A ENVIAR AL JSON..... 7

Solicitud de Gestión a Calculadora..... 7

Respuesta de Calculadora a Gestión..... 7

Cómo se envía y se recibe..... 8

REFERENCIAS..... 9

1. NAMED PIPELINES en C#.

Una pipe sirve para que dos programas se pasen datos. En nuestro caso, tendremos dos aplicaciones WPF que se ejecutan por separado. Como cada una tiene sus propias variables, necesitamos una forma de enviar la información de una a otra. Para eso vamos a usar el pipe.

Usaremos Named Pipes, que significa tuberías con nombre. El nombre sirve para que las dos aplicaciones encuentren el mismo canal. En nuestro caso lo llamaremos pipeMediciones y tendremos que escribirlo igual en las dos aplicaciones. Named Pipes y System.IO.Pipelines no son lo mismo; nosotros vamos a trabajar con Named Pipes.

Cómo funciona

Una de las aplicaciones hará de servidor: creará el pipe y se quedará esperando. La otra hará de cliente y se conectará a él. Al principio puede resultar un poco confuso, pero el servidor del pipe no tiene por qué ser la aplicación que usa la base de datos. Con PipeDirection.InOut podremos enviar y recibir datos por el mismo canal (Microsoft, s. f.-a).

Clases que utilizaremos en C#

Para hacer esto en C# vamos a usar NamedPipeServerStream en el servidor y NamedPipeClientStream en el cliente. Estas clases están en System.IO.Pipes. También necesitaremos StreamReader para leer los mensajes y StreamWriter para escribirlos, que están en System.IO.

```
using System.IO;
using System.IO.Pipes;
```

Con un solo pipe nos basta para enviar los datos y recibir el resultado. Primero escribirá el cliente y leerá el servidor. Después lo harán al revés. Es importante seguir ese orden, porque si los dos se ponen a leer sin que ninguno haya enviado nada, se quedarían esperando.

También tenemos que evitar que la ventana se quede bloqueada mientras espera una conexión o un mensaje. Por eso vamos a usar tareas y async y await en la comunicación. Así podremos seguir usando la ventana mientras la aplicación espera.

2. DOCUMENTACIÓN

En la documentación recogeremos lo que hace el proyecto y cómo lo vamos organizando:

- Objetivo: gestionar un gimnasio y calcular el IMC de sus socios con dos aplicaciones WPF.
- Reparto de funciones: qué hace cada aplicación y qué parte del trabajo realiza cada integrante.
- Diseño: cómo serán las ventanas y qué tablas tendrá la base de datos MySQL, con sus relaciones.
- Comunicación: cómo se llama el pipe, qué aplicación espera, cuál se conecta y qué mensajes se envían.
- Puesta en marcha: qué versión de .NET y qué bibliotecas necesitamos, cómo crear la base de datos y qué aplicación abrir primero.
- Pruebas: qué debería pasar y qué ocurre al probar datos correctos, datos incorrectos, la App 2 cerrada o un corte de conexión.

3. USO EN EL PROYECTO

Nuestro proyecto consiste en dos aplicaciones WPF en C# para un gimnasio. Una se encargará de la gestión y la otra de calcular el IMC. Usaremos el pipe para pasar los datos de la medición de una aplicación a otra y devolver el resultado. Los socios, las clases, las inscripciones, los entrenadores y las tarifas se gestionarán desde la App 1.

Responsabilidad de cada aplicación

App 1, Gestión: será el cliente porque es la que iniciará la conexión. Desde ella guardaremos las mediciones de los socios en la base de datos MySQL Desafio_Neli_Raul. Cuando llegue el resultado de la otra aplicación, lo añadiremos a la medición. Será la única aplicación que accederá a la base de datos.

App 2, Calculadora: será el servidor porque esperará a que la App 1 se conecte. Comprobará que el peso y la altura sean mayores que cero, calculará el IMC y devolverá su clasificación. Esta aplicación no tendrá acceso a la base de datos.

Pasos de una medición

1. Gestión guardará la medición con imc y clasificacion a NULL, es decir, todavía sin resultado, y con estado PENDIENTE. id_solicitud servirá para reconocer cada petición.
2. Gestión se conectará a pipeMediciones y enviará los datos en un mensaje JSON, sin incluir el IMC ni la clasificación.
3. Calculadora leerá el mensaje y comprobará el peso y la altura. Pasará la altura de centímetros a metros y calculará IMC = peso_kg / (altura_m × altura_m).
4. Calculadora devolverá el IMC con un decimal, su clasificación y los códigos que identifican la petición.
5. Gestión comprobará que la respuesta corresponde a esa petición. Después guardará el resultado en la medición y cambiará su estado a CALCULADA.

La clasificación será: menos de 18,5, Bajo peso; desde 18,5 y menos de 25, Normopeso; desde 25 y menos de 30, Sobrepeso; y desde 30, Obesidad. Usaremos el resultado sin redondear para elegir el grupo. El IMC que devolvamos se mostrará con un decimal.

Con ConnectAsync(5000), la App 1 esperará hasta cinco segundos para conectarse a la App 2. Este límite solo sirve para establecer la conexión, no para recibir el resultado. Si no

consigue conectarse, la medición quedará como PENDIENTE o ERROR, según lo que programemos, pero nunca como CALCULADA (Microsoft, s. f.-b).

Para poder hacer más de un cálculo, pondremos el servidor dentro de un bucle. Cuando termine de recibir una solicitud y enviar su resultado, cerrará esa conexión y volverá a esperar otra. Esta es la forma en la que hemos planteado nuestro proyecto, aunque no es la única manera de usar Named Pipes.

4. DATOS A ENVIAR AL JSON

Los datos los enviaremos en JSON, que es un texto escrito con nombres y valores. Por ejemplo, "altura_cm": 180 indica que la altura es de 180 centímetros. El pipe sirve para enviar el mensaje y JSON para escribir los datos que lleva dentro. No necesitamos guardar un archivo .json, ya que mandaremos el texto directamente.

Los datos que tengamos en C# habrá que pasarlos a texto JSON antes de enviarlos. Esto se llama serializar. Al recibirlos, haremos lo contrario para poder trabajar con ellos en el programa: deserializar. Nuestra idea es hacerlo con JsonConvert.SerializeObject y JsonConvert.DeserializeObject de Newtonsoft.Json (Newtonsoft, s. f.-c).

Solicitud de Gestión a Calculadora

```
{
  "id_solicitud": "SOL-0001", "id_medicion": 12,
  "id_socio": 3, "fecha": "2026-10-26T10:30:00",
  "peso_kg": 78.50, "altura_cm": 180,
  "pulsaciones_reposo": 64
}
```

id_solicitud es el código de la petición; id_medicion indica qué medición hemos guardado; e id_socio indica a qué socio pertenece. fecha es el momento de la medición. peso_kg es el peso en kilos y altura_cm, la altura en centímetros. pulsaciones_reposo recoge las pulsaciones, aunque ese dato no se usa para calcular el IMC.

Respuesta de Calculadora a Gestión

```
{
  "id_solicitud": "SOL-0001", "id_medicion": 12,
  "imc": 24.2, "clasificacion": "Normopeso",
  "estado": "CALCULADA"
}
```

En la respuesta mantendremos los códigos de la petición y añadiremos el IMC, la clasificación y el estado. Los ejemplos están separados en varias líneas para que se vean mejor, pero al enviarlos usaremos Formatting.None. De esta forma, cada mensaje irá en una sola línea (Newtonsoft, s. f.-b).

Cómo se envía y se recibe

```
string json = JsonConvert.SerializeObject(
    solicitud, Formatting.None);
await writer.WriteLineAsync(json);
string recibido = await reader.ReadLineAsync();
var respuesta = JsonConvert.DeserializeObject<RespuestaMedicion>(recibido);
```

Para enviar el mensaje usaremos WriteLineAsync, que añade un salto de línea al final. La otra aplicación lo leerá con ReadLineAsync hasta llegar a ese salto, y así sabrá que el mensaje ha terminado (Microsoft, s. f.-c). También pondremos AutoFlush = true para que el texto se envíe al escribirlo y no se quede guardado temporalmente mientras la otra aplicación espera recibirlo (Microsoft, s. f.-d).

En las dos aplicaciones usaremos las clases SolicitudMedicion y RespuestaMedicion con los mismos datos. Si una propiedad se llama PesoKg en C# y peso_kg en JSON, pondremos [JsonProperty("peso_kg")] para relacionar los dos nombres (Newtonsoft, s. f.-a). También tendremos que comprobar qué pasa si llega un mensaje incorrecto, no recibimos respuesta o se corta la conexión, antes de guardar el resultado en la base de datos.

REFERENCIAS

Chacon, S., & Straub, B. (2014). *What is Git?* En Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

Git. (s. f.). *gitignore*.
https://git-scm.com/docs/gitignore

Microsoft. (s. f.-a). *How to: Use named pipes for network interprocess communication*. Microsoft Learn.
https://learn.microsoft.com/en-us/dotnet/standard/io/how-to-use-named-pipes-for-network-interprocess-communication

Microsoft. (s. f.-b). *NamedPipeClientStream.ConnectAsync method*. Microsoft Learn.
https://learn.microsoft.com/en-us/dotnet/api/system.io.pipes.namedpipeclientstream.connectasync

Microsoft. (s. f.-c). *StreamReader.ReadLine method*. Microsoft Learn.
https://learn.microsoft.com/en-us/dotnet/api/system.io.streamreader.readline

Microsoft. (s. f.-d). *StreamWriter.AutoFlush property*. Microsoft Learn.
https://learn.microsoft.com/en-us/dotnet/api/system.io.streamwriter.autoflush

Newtonsoft. (s. f.-a). *JsonPropertyAttribute name*. Json.NET Documentation.
https://www.newtonsoft.com/json/help/html/JsonPropertyName.htm

Newtonsoft. (s. f.-b). *Serialize unindented JSON*. Json.NET Documentation.
https://www.newtonsoft.com/json/help/html/SerializeUnindentedJson.htm

Newtonsoft. (s. f.-c). *Serializing and deserializing JSON*. Json.NET Documentation.
https://www.newtonsoft.com/json/help/html/SerializingJSON.htm